

Bàn về bài toán dạng nộp kết quả

Lei Huanzhong

2003

Nguồn gốc: On output-only problems

IOI China candidate team papers / 2003 / Lei Huanzhong / Lei Huanzhong.doc

Abstract

Bài toán dạng nộp kết quả có lịch sử còn rất ngắn, nhưng nhờ mô hình mới, cách nhìn mới về hiệu quả, thời gian và bộ nhớ, cùng các kỹ thuật giải độc đáo, nó nhanh chóng trở thành một dạng bài quan trọng trong Olympic Tin học. Bài viết thảo luận hai kỹ thuật chính của dạng bài này: phân tích dữ liệu và ngẫu nhiên hóa. Nguyên tắc cốt lõi có thể tóm lại bằng một câu: trong thời gian ngắn nhất, hãy lấy được kết quả đúng; điều quan trọng không phải là quá trình.

Từ khóa: nộp kết quả, phân tích dữ liệu, ngẫu nhiên hóa, chiến lược.

1 Dẫn nhập

Bài toán dạng nộp kết quả là một trong những dạng bài trẻ nhất của Olympic Tin học. Việc IOI liên tiếp xuất hiện dạng bài này trong hai năm cho thấy nó đã bước vào các kỳ thi chính thống. Lý do quan trọng là dạng bài này linh hoạt, mới mẻ, khuyến khích nhiều con đường khác nhau đi tới cùng một kết quả, và tạo không gian cho những phương pháp khác thường.

Nhìn lại các bài như *Crack the Code*, *Double Crypt*, *Birthday Party*, *Tetris* và *Xor*, ta thấy mỗi bài đều có ý tưởng thú vị, và phương pháp giải cũng rất đa dạng. Nhưng muốn làm đúng và nhanh dạng bài này, ta cần suy nghĩ sâu hơn về chính đặc điểm của nó.

Sự xuất hiện của bài nộp kết quả làm thay đổi nhiều thứ.

Thay đổi nội dung khảo sát. Khi Olympic Tin học đã phát triển tương đối chín, nếu chỉ khảo sát thuật toán và cấu trúc dữ liệu thì chưa đủ. Bài nộp kết quả còn khảo sát năng lực tư duy, khả năng sáng tạo, chiến lược tổng thể và chiến lược thời gian. Những yếu tố này cũng tồn tại trong các bài truyền thống, nhưng ở dạng nộp kết quả chúng được thể hiện trực tiếp hơn nhiều.

Thay đổi độ phân hóa. Với bài truyền thống, ta thường so sánh thuật toán bằng độ phức tạp thời gian và bộ nhớ. Nhưng ở bài nộp kết quả, điều quan trọng hơn là thời gian để tạo ra các tệp đáp án đúng. Nếu hai người đều tạo đáp án đúng, người hoàn thành trong 20 phút rõ ràng thắng người mất 2 giờ, bất kể họ dùng phương pháp nào.

Thay đổi bản chất đề bài. Một bài như *Birthday Party* nếu được ra dưới dạng nộp chương trình thì cách nghĩ sẽ khác hẳn so với dạng nộp kết quả. Khi tác giả đề bài chọn dạng nộp kết quả, họ không muốn thí sinh chỉ nhìn nó như một bài chuẩn, mà muốn thí sinh tận dụng chính dữ liệu đầu vào đã cho, mạnh dạn dùng mọi cách để thu được kết quả đúng nhanh nhất.

Thay đổi cách chấm. Chấm bài lập trình vốn đã là hộp đen; với bài nộp kết quả, “hộp đen” càng nhấn mạnh vào kết quả cuối cùng. Chương trình hay quá trình sinh ra đáp án không còn quan trọng bằng việc tệp nộp có đúng hay không.

Vì vậy, nguyên tắc giải bài có thể viết ngắn gọn:

Luôn nhắc mình rằng thứ cần có là kết quả đúng trong thời gian ngắn nhất, không phải một quá trình hoàn hảo.

2 Phân tích dữ liệu

Phân tích dữ liệu gần như sinh ra cùng với bài nộp kết quả. Nó đòi hỏi khả năng quan sát, phỏng đoán, thực nghiệm và thao tác thủ công cao hơn nhiều so với các phương pháp thông thường. Phần này dùng hai ví dụ: *Crack the Code* và *Birthday Party*.

2.1 *Crack the Code* (Baltic OI 2001)

Tóm tắt đề. Có năm cặp tệp: mỗi tệp `cra*.txt` là một văn bản gốc tiếng Anh, và tệp `cra*.in` tương ứng được tạo từ một tệp chưa biết `cra*.out`. Với mỗi tệp, có 10 số kiểu byte chưa biết. Khi xử lý ký tự thứ i , nếu ký tự không phải chữ cái thì giữ nguyên; nếu là chữ thường thì đổi thành chữ hoa; nếu là chữ hoa thì dịch vòng trong bảng chữ cái theo một số bước phụ thuộc vào $i \bmod 10$. Nhiệm vụ là từ `cra*.in` và `cra*.txt` khôi phục `cra*.out`.

Bài này không có một thuật toán thật sự hiệu quả cho mọi dữ liệu, vì bản chất là một dạng mã hóa khá hoàn chỉnh. Đó chính là lý do nó phù hợp với hình thức nộp kết quả: chỉ cần phá được các dữ liệu đã cho.

Dùng dữ liệu để phá mã. Điểm đầu tiên cần chú ý là chỉ có 5 bộ dữ liệu, và mỗi văn bản đều là tiếng Anh. Trong tiếng Anh, nhiều từ tần suất cao như *a, I, the, he, are, of, in, after* rất ngắn; từ càng ngắn thì khả năng càng ít.

Mục tiêu chính là tìm 10 số dịch chuyển. Nếu xác định được một vài cặp từ tương ứng giữa bản rõ và bản mã, các dịch chuyển sẽ lộ ra ngay.

Trong dữ liệu thứ nhất, đoạn mã có cụm EQMDY 1943. Dựa vào văn bản gốc và ngữ cảnh tiểu sử, gần như chắc chắn EQMDY là AFTER. Chỉ từ một từ này đã suy ra được 5 trong 10 số dịch chuyển. Sau khi thử khôi phục bằng 5 số đó, nhiều cụm từ tiếp theo hiện ra gần đúng, chẳng hạn THG NXHPARD UNKBBHMITY rõ ràng là THE HARVARD UNIVERSITY, và UNIVETYFJS OF CALKLLHHIA là UNIVERSITY OF CALIFORNIA. Dùng thêm một hai cụm dài như vậy là đủ khôi phục toàn bộ văn bản.

Trong dữ liệu thứ năm, cách thủ công còn nhanh hơn. Văn bản gốc nói về một không gian vectơ tô pô; dòng mã bắt đầu bằng một từ 10 chữ cái kèm dấu hai chấm. Rất tự nhiên đoán đó là DEFINITION. Thử đoán này thì toàn bộ đoạn trở nên rõ:

DEFINITION: THE DUAL SPACE OF A TOPOLOGICAL VECTOR SPACE X
IS THE VECTOR SPACE X^* WHOSE ELEMENTS ARE THE CONTINUOUS
LINEAR FUNCTIONALS ON X .

Ở đây, trực giác ngôn ngữ và tri thức nền của con người đóng vai trò quyết định. Nếu các bài truyền thống thường khảo sát thí sinh thông qua chương trình, thì dạng bài này gần như đối thoại trực tiếp với thí sinh.

Phương pháp thống kê tần suất. Ta cũng có thể dùng chương trình. Vì văn bản tiếng Anh chỉ có 26 chữ cái, tần suất xuất hiện của mỗi chữ không đều. Gọi P là phân bố tần suất chữ cái trong văn bản gốc. Với mỗi lớp vị trí $i, i + 10, i + 20, \dots$ trong văn bản mã, tính phân bố H , rồi thử mọi dịch chuyển $0, \dots, 25$ và chọn dịch chuyển làm H giống P nhất.

Bài gốc so sánh vài hàm đánh giá mức giống nhau. Kết quả cho thấy khi mẫu đủ dài, phương pháp thống kê khá tốt; khi văn bản quá ngắn, tần suất mất ý nghĩa và phương pháp thất bại. Khi đó dữ liệu phân tích thủ công lại có ưu thế.

Điều rút ra là: với bài nộp kết quả, không có phương pháp nào luôn tốt nhất. Phân tích dữ liệu, chương trình phụ trợ ngắn, thống kê tần suất và kiểm tra thủ công thường phải phối hợp với nhau.

2.2 *Birthday Party* (CEOI 2002)

Tóm tắt đề. Có N người và M yêu cầu lô-gic. Một yêu cầu có thể là tên một người, phủ định của tên đó, phép $\wedge$, phép $\vee$, hoặc phép kéo theo $\Rightarrow$ giữa các yêu cầu. Với 10 tệp `party*.in`, cần chọn một tập người sao cho mọi yêu cầu đều đúng. Đề bảo đảm có ít nhất một nghiệm và chỉ cần nộp một nghiệm. Kích thước có thể tới

$$N \leq 25000, \quad M \leq 129998.$$

Nhìn hình thức, đây là SAT, vốn là bài toán NP-đầy đủ. Quy mô dữ liệu cho thấy nếu giải được thì mẫu chốt chắc chắn nằm ở dữ liệu cụ thể, không nằm ở một thuật toán tổng quát cho SAT.

Phân tích từng dữ liệu. Dữ liệu đầu tiên rất nhỏ, có thể giải bằng tay hoặc bằng chương trình đơn giản. Với dữ liệu thứ hai và thứ ba, biểu thức hầu hết có dạng mệnh đề OR ngắn; tên biến lại có quy luật rất rõ: $a, b, c, \dots, j, ba, bb, \dots$ tương ứng tự nhiên với các số liên tiếp. Chỉ cần một chương trình quét dữ liệu là có thể phát hiện quy luật này và xử lý thuận lợi hơn.

Dữ liệu thứ tư có vài dòng biểu thức phức tạp hơn, nhưng số dòng đặc biệt rất ít. Một chiến lược hợp lý là dùng chương trình xử lý phần lớn mệnh đề đơn giản, rồi kiểm tra thủ công sáu yêu cầu đặc biệt. Viết một bộ phân tích cú pháp đầy đủ chỉ để phục vụ sáu dòng như vậy thường không đáng.

Dữ liệu thứ năm đến thứ bảy hầu như toàn là dạng

$$A \Rightarrow B,$$

trong đó A, B là biến hoặc phủ định biến. Đây là 2-SAT dưới dạng đồ thị hàm ý. Dữ liệu thứ tám và thứ chín hầu như toàn là dạng

$$A \vee B,$$

cũng có thể chuyển về 2-SAT. Dữ liệu thứ chín còn có vài dòng đặc biệt, nhưng số lượng rất ít và có thể xử lý riêng. Dữ liệu thứ mười ban đầu có vẻ rất đáng sợ vì biểu thức không đều, nhưng quan sát phần cuối sẽ thấy nhiều ràng buộc dạng

$$x \Rightarrow \neg x, \quad \neg y \Rightarrow y,$$

ép trực tiếp giá trị của rất nhiều biến. Phần còn lại vì thế nhỏ hơn nhiều.

Bài học ở đây là: dữ liệu không chỉ là đầu vào, mà còn là gợi ý chiến lược. Nếu không quan sát kỹ, ta dễ bỏ qua vài dòng đặc biệt hoặc vài quy luật đặt tên biến; nhưng chính những chi tiết ấy quyết định cách giải nhanh nhất.

So sánh với bài thường. Nếu *Birthday Party* là bài nộp chương trình, ta phải xây dựng một bộ xử lý biểu thức lô-gic khá tổng quát. Nhưng với bài nộp kết quả, có thể kết hợp nhiều thủ đoạn: giải tay dữ liệu nhỏ, dùng 2-SAT cho dữ liệu có cấu trúc, viết chương trình quét để phát hiện dòng đặc biệt, và kiểm tra thủ công vài biểu thức ngoại lệ.

3 Ngẫu nhiên hóa

Ngẫu nhiên hóa cũng rất phù hợp với bài nộp kết quả, vì ta có thể chạy nhiều lần, chọn kết quả tốt nhất, và không cần chứng minh quá trình luôn ổn định như trong bài nộp chương trình. Hai ví dụ tiêu biểu là *Tetris* và *Xor*.

3.1 Tetris

Trong bài *Tetris*, mỗi dữ liệu cho một trạng thái ban đầu và cần tạo một chuỗi thao tác đặt khối để làm phẳng bàn chơi. Một số dữ liệu nhỏ có thể tính tay, một số có quy luật rõ chỉ cần chương trình ngắn, còn các dữ liệu lớn không thể làm thủ công.

Một thuật toán khả thi là dùng vài dạng khối “dễ chịu” để trước hết đưa bàn về trạng thái thấp, sau đó dùng ngẫu nhiên hóa để lấp các chỗ trống từ trái sang phải. Mỗi lần gặp một vị trí cần xử lý, chọn ngẫu nhiên một khối hợp lệ để lấp. Thuật toán này thô nhưng cài rất nhanh.

Sau đó có thể thêm một số kinh nghiệm thủ công: khi bàn đã khá phẳng, ưu tiên các khối làm bề mặt tiếp tục phẳng; tránh các khối tạo ra độ nhô đột ngột; ưu tiên vài khối đặc biệt nếu chúng làm giảm chiều cao tổng thể. Những quy tắc này không phải chứng minh chặt, nhưng trong bài nộp kết quả chỉ cần chúng tạo đáp án đúng cho dữ liệu đã cho.

Bài gốc còn nêu hai phương pháp ổn định hơn:

- Mỗi bước chọn thao tác làm bàn phẳng nhất theo một hàm đánh giá như tổng độ lệch giữa chiều cao các cột.
- Chia bàn thành nhóm 4 cột, xử lý từng nhóm bằng cấu trúc xây dựng, rồi ghép các nhóm lại.

Phương pháp xây dựng có chất lượng tốt và chạy nhanh, nhưng cài phức tạp hơn. Ngược lại, ngẫu nhiên hóa dễ viết, đủ nhanh để thử nhiều lần, và phù hợp với mục tiêu lấy kết quả trong thời gian ngắn.

3.2 Xor (IOI 2002)

Bài *Xor* cho một bảng $N \times N$ ô đen trắng. Mỗi thao tác chọn một hình chữ nhật và đảo màu mọi ô trong đó. Cần dùng càng ít thao tác càng tốt để biến toàn bộ bảng thành trắng. Mỗi dữ liệu được tính điểm theo số thao tác.

Kinh nghiệm cho thấy nên nhìn vào các “góc” của vùng đen. Có thể chia góc thành ba loại; góc loại một và hai chắc chắn là góc của một hình chữ nhật trong một lời giải nào đó, còn góc loại ba thì không nhất thiết. Thuật toán tham lam cơ bản:

1. Tạo tập A gồm mọi góc loại một và hai.
2. Nếu trong A có bốn điểm là bốn đỉnh của một hình chữ nhật, xem đó là một thao tác và xóa bốn điểm khỏi A .
3. Nếu không, chọn ba điểm tạo thành ba đỉnh của một hình chữ nhật, thêm đỉnh thứ tư còn thiếu và thực hiện thao tác tương ứng; cập nhật A .
4. Lặp đến khi A rỗng.

Phản xóa bốn điểm không phụ thuộc nhiều vào thứ tự, nhưng phần chọn ba điểm có ảnh hưởng rõ tới chất lượng lời giải. Vì $N \leq 2000$, thử một chiến lược chọn phức tạp có thể tốn thời gian; do đó có thể chọn ngẫu nhiên nhiều lần và lấy lời giải tốt nhất. Thực nghiệm trong bài gốc cho thấy sau nhiều lần chạy, kết quả rất gần lời giải tốt nhất đã biết, một số dữ liệu còn đạt mức tối ưu đã biết.

Đây là một ưu thế đặc thù của nộp kết quả: ta không cần một thuật toán có hiệu năng ổn định trên mọi lần chạy; ta chỉ cần có một lần chạy sinh ra tệp đáp án tốt.

4 Chiến lược cho bài nộp kết quả

Dạng bài này không chỉ khảo sát thuật toán, mà còn khảo sát chiến lược thi.

4.1 Cách nhìn mới về hiệu quả

Trong bài truyền thống, ta thường hình dung quá trình làm bài như sau: ban đầu mất thời gian thiết kế thuật toán và cấu trúc dữ liệu, sau đó chương trình mới dần đạt hiệu quả cao. Có những lúc ta tiếp tục tối ưu thuật toán dù kết quả hiện tại đã đủ để qua bài; về chiến lược thi, đó có thể là lãng phí.

Với bài nộp kết quả, đường cong thường khác: ban đầu có thể lấy điểm rất nhanh bằng phân tích dữ liệu, giải tay hoặc chương trình phụ trợ nhỏ; về sau mỗi điểm tăng thêm càng tốn công hơn. Vì dữ liệu đã cho trước, ta có thể ước lượng khá chính xác mình còn lấy được bao nhiêu điểm trong bao lâu. Do đó cần quyết định khi nào nên dừng, khi nào nên bỏ một vài dữ liệu khó để dành thời gian cho bài khác, và khi nào nên viết một chương trình tổng quát hơn.

4.2 Cách nhìn mới về thời gian và bộ nhớ

Bài nộp kết quả phá vỡ nhiều thói quen của bài truyền thống:

- bộ nhớ dùng khi sinh đáp án thường rộng rãi hơn;
- thời gian chạy có thể dài hơn nhiều, thậm chí vài giờ nếu cuộc thi cho phép;
- có thể chạy nhiều tiến trình hoặc nhiều thử nghiệm song song;
- dữ liệu đầu vào cố định, nên có thể khai thác mọi đặc điểm riêng của nó.

Điều này không có nghĩa bỏ qua thuật toán đẹp, mà là phải kết hợp thực lực với mẹo phù hợp. Mục tiêu là tệp kết quả đúng, không phải một chương trình đẹp cho mọi dữ liệu tưởng tượng.

4.3 *Double Crypt* (IOI 2001)

Bài *Double Crypt* dùng mã hóa AES hai lần:

$$c = E(E(p, k_1), k_2),$$

với k_1, k_2 nằm trong một tập khóa giới hạn bởi tham số s . Cần tìm k_1, k_2 từ p, c, s .

Thuật toán chuẩn là meet-in-the-middle: tính mọi $E(p, k_1)$, lưu vào bảng băm; đồng thời tính mọi $D(c, k_2)$ và tìm giao. Nếu bỏ qua va chạm, thời gian và bộ nhớ đều khoảng $O(2^s)$. Thuật toán này rõ ràng tốt nhưng cần cài đặt cẩn thận.

Trong bối cảnh nộp kết quả, có thể tạm dùng cách liệt kê đơn giản hơn cho một số dữ liệu, tận dụng các đặc điểm như dữ liệu thứ ba và thứ chín có cùng p, c , hoặc một số dữ liệu có thể chạy chung một lần. Dù chương trình chạy lâu hơn, ta có thể để nó chạy nền trong khi làm bài khác. Cách này không hoàn hảo, nhưng có thể nhanh chóng lấy phần lớn điểm, phù hợp với chiến lược tổng thể.

4.4 Phối hợp tính tay và chương trình

Nhiều bài nộp kết quả hay ở chỗ buộc ta phối hợp tính tay với chương trình. *Crack the Code* và *Birthday Party* đều như vậy. Nếu quá lệ thuộc vào tính tay, các dữ liệu lớn sẽ không làm nổi; nếu quá lệ thuộc vào chương trình tổng quát, ta mất thời gian cài những thứ không cần thiết cho vài dòng ngoại lệ.

Người giải giỏi là người biết dùng cả bộ não lẫn máy tính: dùng chương trình ngắn để quét, thống kê, kiểm tra; dùng mắt người để nhận ra quy luật, đoán từ khóa, xử lý ngoại lệ; rồi lại dùng chương trình để xác minh và xuất tệp.

5 Kết luận

Bài toán dạng nộp kết quả buộc ta rời khỏi cách nhìn truyền thống. Phải phân tích từng dữ liệu cụ thể, thiết kế phương pháp từ góc nhìn toàn cục, và phối hợp linh hoạt giữa thuật toán, quan sát, thử nghiệm, ngẫu nhiên hóa, tính tay và chương trình phụ trợ.

Cùng một bài toán, nếu là nộp chương trình thì có thể cần một thuật toán ổn định, tổng quát; nếu là nộp kết quả thì phương pháp tốt nhất có thể là một tổ hợp các thủ đoạn nhỏ. Tiêu chuẩn đánh giá đã thay đổi, nên chiến lược cũng phải thay đổi.

Tóm lại: hãy luôn nhắc mình rằng thứ cần có là kết quả đúng trong thời gian ngắn nhất, không phải quá trình.

References

- [1] Marcin Kubica. *The 7th Baltic Olympiad in Informatics BOI 2001 Tasks and Solutions*.
- [2] *Summary of the 9th Central European Olympiad in Informatics*.
- [3] *Problems of the 19th National Olympiad in Informatics, 2002*.
- [4] *IOI 2002 Problems*.
- [5] Jyrki Nummenmaa, Erkki Makinen, Isto Aho. *IOI'01 Competition*, second edition.